



Sistema de Asistencias Vehiculares — Prueba AsisYa

Asignación inteligente de proveedores

Entregables 4.1 · 4.2 · 4.3 · 4.4

Squad PruebaAsisYa · 2026

De una solicitud de asistencia a una asignación trazable

OBJETIVO

- Recibir solicitudes desde la app.
- Validar ubicación y disponibilidad.
- Encontrar y asignar el proveedor óptimo.
- Permitir seguimiento en tiempo real.

ALCANCE

- Arquitectura C4 niveles 1–3.
- Microservicio crítico en .NET.
- Estándares técnicos del squad.
- Ejercicio práctico de Code Review.

VALOR ESPERADO

- Menores tiempos de atención.
- Mayor tasa de aceptación.
- Mayor trazabilidad operativa.
- Base preparada para evolución.

El reto no es solo asignar: es asignar rápido, correctamente y con control



Requerimientos clave

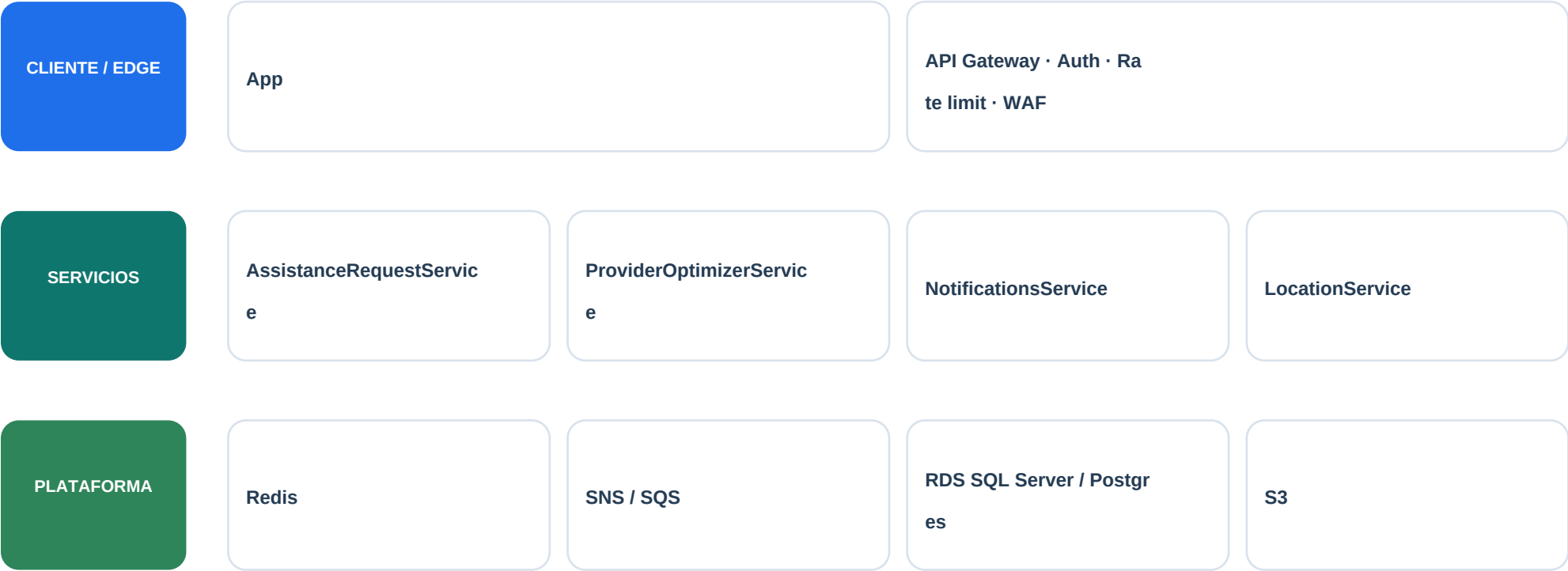
- Latencia baja y tolerancia a picos de demanda.
- Seguridad, observabilidad y capacidad de evolución.

Situación a resolver

Asignación manual o por reglas simples

→ demoras · asignaciones no óptimas ·

Arquitectura desacoplada, escalable y observable



Cross-cutting: observabilidad · métricas · trazas · logs · IAM · JWT · retries · autoscaling

El componente crítico concentra la lógica de selección



ENDPOINTS

- POST /optimize · inicia selección para requestId
- GET /providers/available · consulta disponibilidad

OPERACIÓN LOCAL

- SQL Server + scripts de init
- Dockerfile + docker-compose

CALIDAD

- Tests unitarios
- Tests de integración · InMemory / WebApplicationFactory

Un mismo marco para construir, revisar y desplegar

Código

● .NET · React · Docker

Flujo

● Branch strategy · Code Reviews

Entrega

● CI/CD · Definition of Done

Seguridad

● Secretos · buenas prácticas

Arquitectura

● Patrones y convenciones base

Deuda técnica

● Registro y control continuo

Documento de referencia: <docs/4.3/squad-technical-standards.md>

El ejercicio prueba criterio técnico, no solo sintaxis

Concurrencia

1

Condiciones de carrera · optimistic locking

Persistencia

2

Transacciones y consistencia

Diseño

3

SOLID · separación de responsabilidades · DTOs

Asincronía

4

Uso correcto de operaciones async

Validación

5

Datos de entrada y reglas

Resiliencia

6

Circuit breakers y mitigaciones

Automatizar calidad antes de desplegar



Los principales riesgos ya tienen controles propuestos

Datos incompletos Ubicación / estado del proveedor	MITIGACIÓN	Validación · enriquecimiento · fallbacks
Picos de demanda Saturación y aumento de latencia	MITIGACIÓN	SQS · autoscaling · throttling
Seguridad Exposición de secretos	MITIGACIÓN	Secrets Manager · IAM least privilege

De la prueba técnica hacia una validación operativa

01

Diagramas renderizados

Adjuntar C4 visual a la presentación.

02

E2E + carga

Validar con datos reales y medir comportamiento.

03

ETA reales

Integrar proveedor de mapas: AWS Location / Google Maps.

04

Optimización

Ajustar política con métricas reales; A/B testing y ML opcional.

Ejecución local y recursos del proyecto

LOCAL

- dotnet restore
- dotnet build
- docker-compose up --build
- dotnet test

RECURSOS

- docs/4.1/ · arquitectura
- ProviderService/ · microservicio
- docs/4.3/ · estándares
- CodeReview/ · snippet

PRESENTACIÓN

- Renderizar diagramas C4
- Completar pruebas E2E
- Mantener métricas de calidad
- Usar notas del orador

Fuente: contenido entregado en presentation(1).md. La presentación fue reorganizada y visualmente sintetizada sin agregar funcionalidades no documentadas.